

Procedimentos Armazenados

QXD0011 - Fundamentos de Bancos de Dados

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Procedimentos armazenados
- Utilidade
- Formato geral
- Chamadas
- Parâmetros
- PL/pgSQL - Procedural Language/PostgreSQL

Procedimentos armazenados

- Motivação
 - Às vezes é útil criar módulos de programa de banco de dados – procedimentos ou funções – que são armazenados e executados pelo SGBD no próprio servidor de banco de dados.
- Definição
 - Módulos de programa armazenados pelo SGBD no servidor de dados.
- Termo no padrão SQL: módulos armazenados persistentes.
 - São armazenados persistentemente pelo SGBD.

Utilidade

- Compartilhamento para uso por várias aplicações.
- A execução no servidor pode reduzir a transferência de dados e o custo de comunicação entre cliente e servidor.
- Podem permitir que tipos mais complexos de dados derivados estejam disponíveis.
- Podem ser usados para verificar restrições complexas que estão além do poder das asserções e triggers.

Formato geral

```
CREATE PROCEDURE <nome do procedimento>  
(<parametros>)  
<declaracoes de local>  
<corpo do procedimento> ;
```

```
CREATE FUNCTION <nome da funcao>  
(<parametros>)  
RETURNS <tipo de retorno>  
<declaracoes de local>  
<corpo da funcao> ;
```

Chamadas

- É possível chamar procedimentos e funções armazenados das várias interfaces SQL e linguagens de programação.
- A instrução CALL do padrão SQL pode ser usada para chamar um procedimento armazenado:
 - CALL <nome do procedimento ou função>(<lista de argumentos>);
- Se essa instrução for chamada da JDBC, ela deve ser atribuída a um objeto de instrução do tipo CallableStatement.

Parâmetros

- Em geral, cada parâmetro deve ter um tipo de parâmetro, o qual é um dos tipos de dados da SQL.
- Cada parâmetro também deve ter um modo de parâmetro, que é um entre:
 - IN (entrada), OUT (saída) e INOUT (entrada e saída) .

PL/pgSQL - Procedural Language/PostgreSQL

- PostgreSQL usa o Bison como parser, de modo que é fácil portar linguagens open source, bem como reusar código.
- Linguagens procedurais disponíveis para o PostgreSQL:
 - PL/pgSQL
 - PL/Java
 - PL/Perl
 - PL/pgPSM
 - PL/php
 - PL/Python
 - PL/R
 - PL/Ruby
 - PL/sh
 - PL/Tcl
 - PL/Lua.

PL/pgSQL

- Linguagem procedural carregável desenvolvida para o PostgreSQL.
- Objetivos
 - criar procedimentos de funções e de gatilhos;
 - adicionar estruturas de controle à linguagem SQL;
 - realizar processamentos complexos;
 - herdar todos os tipos de dado, funções e operadores definidos pelo usuário;
 - ser definida como confiável pelo servidor;
 - ser fácil de usar.

Formato geral no PostgreSQL

CREATE [OR REPLACE] FUNCTION

```

name ( [ [ argmode ] [ argname ] argtype [ { DEFAULT | = } default_expr ] [, ...] ] )
[ RETURNS rettype
  | RETURNS TABLE ( column_name column_type [, ...] ) ]
{ LANGUAGE lang_name
  | WINDOW
  | IMMUTABLE | STABLE | VOLATILE
  | CALLED ON NULL INPUT | RETURNS NULL ON NULL INPUT | STRICT
  | [ EXTERNAL ] SECURITY INVOKER | [ EXTERNAL ] SECURITY DEFINER
  | COST execution_cost
  | ROWS result_rows
  | SET configuration_parameter { TO value | = value | FROM CURRENT }
  | AS 'definition'
  | AS 'obj_file', 'link_symbol'
} ...
[ WITH ( attribute [, ...] ) ]

```

PL/pgSQL

```
CREATE FUNCTION populate() RETURNS integer AS
$$
DECLARE
    -- declarações
BEGIN
    PERFORM minha_funcao();
END;
$$ LANGUAGE plpgsql;
```

PL/pgSQL

- O BEGIN/END da linguagem PL/pgSQL é apenas para agrupamento; não começam nem terminam transações.
- Os procedimentos de funções e de gatilhos são sempre executados dentro da transação estabelecida pelo comando externo.
 - Não podem iniciar ou efetivar transações, porque não haveria contexto para estas serem executadas.

Bloco

```
[ <<rótulo>> ]
[ DECLARE
    declarações ]
BEGIN
    instruções
END;
```

Exemplo

```
CREATE FUNCTION func_escopo() RETURNS integer AS $$
DECLARE
    quantidade integer := 30;
BEGIN
    RAISE NOTICE 'Aqui a quantidade é %', quantidade; -- A quantidade aqui é 30
    quantidade := 50;
    --
    -- Criar um sub-bloco
    --
    DECLARE
        quantidade integer := 80;
    BEGIN
        RAISE NOTICE 'Aqui a quantidade é %', quantidade; -- A quantidade aqui é 80
    END;

    RAISE NOTICE 'Aqui a quantidade é %', quantidade; -- A quantidade aqui é 50

    RETURN quantidade;
END;
$$ LANGUAGE plpgsql;
```

Executando o exemplo

```
SELECT func_escopo();
```

Resultado:

NOTA: Aqui a quantidade é 30

NOTA: Aqui a quantidade é 80

NOTA: Aqui a quantidade é 50

func_escopo

50

(1 linha)

Exemplo

```
CREATE FUNCTION taxa_de_venda(subtotal real) RETURNS real AS $$  
BEGIN  
    RETURN subtotal * 0.06;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION taxa_de_venda(real) RETURNS real AS $$  
DECLARE  
    subtotal ALIAS FOR $1;  
BEGIN  
    RETURN subtotal * 0.06;  
END;  
$$ LANGUAGE plpgsql;
```

Exemplo

```
CREATE FUNCTION somar_tres_valores(v1 anyelement, v2 anyelement,
v3 anyelement)
RETURNS anyelement AS $$
DECLARE
    resultado ALIAS FOR $0;
BEGIN
    resultado := v1 + v2 + v3;
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;
```

Condicionais

```

IF expressão_booleana THEN
    instruções
[ ELSIF expressão_booleana THEN
    instruções
[ ELSIF expressão_booleana THEN
    instruções
    ...]]
[ ELSE
    instruções ]
END IF;

```

Condicionais

```

IF linha_demo.sexo = 'm' THEN
    sexo_extenso := 'masculino';
ELSE
    IF linha_demo.sexo = 'f' THEN
        sexo_extenso := 'feminino';
    END IF;
END IF;

```

Laços

Realizados através das instruções LOOP, EXIT, WHILE e FOR.

```
[<<rótulo>>]  
LOOP  
    instruções  
END LOOP;
```

```
[<<rótulo>>]  
WHILE expressão LOOP  
    instruções  
END LOOP;
```

```
[<<rótulo>>]  
FOR nome IN [ REVERSE ] expressão .. expressão LOOP  
    instruções  
END LOOP;
```

```
EXIT [ rótulo ] [ WHEN expressão ];
```

Laços

LOOP

```
-- algum processamento  
IF contador > 0 THEN  
    EXIT; -- sair do laço  
END IF;  
END LOOP;
```

LOOP

```
-- algum processamento  
EXIT WHEN contador > 0; -- mesmo resultado do exemplo acima  
END LOOP;
```

BEGIN

```
-- algum processamento  
IF estoque > 100000 THEN  
    EXIT; -- causa a saída do bloco BEGIN  
END IF;  
END;
```

Laços

```
WHILE quantia_devida > 0 AND  
saldo_do_certificado_de_bonus > 0 LOOP  
    -- algum processamento  
END LOOP;  
  
WHILE NOT expressão_booleana LOOP  
    -- algum processamento  
END LOOP;
```

Exemplo: Soma condicional

```
CREATE OR REPLACE FUNCTION soma_maiores(IN nomeAtrib TEXT, IN valor
NUMERIC)
RETURNS NUMERIC AS $$
DECLARE
atrib_value NUMERIC;
soma NUMERIC := 0;
BEGIN
FOR atrib_value IN EXECUTE format('SELECT %I FROM FUNCIONARIO WHERE %I >
%L', quote_ident(nomeAtrib), quote_ident(nomeAtrib), valor) LOOP
    soma := soma + atrib_value;
END LOOP;
RETURN soma;
END;
$$ LANGUAGE plpgsql;
```

Bibliografia Básica

- ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 6 ed. Pearson/Addison-Wesley, 2011. ISBN: 9788579360855
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. Rio de Janeiro: Elsevier: Campus, 2012. ISBN 9788535245356
- HEUSER, C. A. Projeto de banco de dados. 6 ed. Bookman, 2009. ISBN: 9788577803828



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

